arXiv:2411.05288v2 [cs.DC] 5 May 2025

BALANCING PIPELINE PARALLELISM WITH VOCABULARY PARALLELISM

Man Tsung Yeung [† 1 2] **Penghui Qi** [1 2] **Min Lin** [1] **Xinyi Wan** [1]

ABSTRACT

Pipeline parallelism is widely used to scale the training of transformer-based large language models, various works have been done to improve its throughput and memory footprint. In this paper, we address a frequently overlooked issue: the vocabulary layers can cause imbalanced computation and memory usage across pipeline stages, worsening pipeline bubbles and the memory bottleneck. To tackle this, we partition the vocabulary layers evenly across pipeline devices and group the computation into pipeline passes. To reduce the activation memory overhead, we propose several algorithms to reduce communication barriers within vocabulary layers. Additionally, we utilize a generalizable method to integrate Vocabulary Parallelism with existing pipeline schedules. By combining these techniques, our methods effectively balance the computation and parameter memory, with only a small constant activation memory overhead. Notably, when combined with activation memory-balanced schedules like *V-Half*, our approach achieves perfect balance in both memory and computation. Extensive evaluations demonstrate that our method achieves computation and memory balance regardless of the vocabulary size, resulting in a 5% to 51% improvement in throughput compared to naïve approaches, meanwhile significantly reducing peak memory usage especially for large vocabulary scenarios. Our implementation is open-sourced at https://github.com/sail-sg/VocabularyParallelism.

1 INTRODUCTION

As the scale of transformer models (Vaswani et al., 2017; Brown et al., 2020) continues to grow, model parallelism has garnered significant attention within the deep learning community. Several model parallel techniques have been proposed to address the challenges associated with training large models, including Zero Redundancy Optimizer (ZeRO) (Rajbhandari et al., 2020; Zhao et al., 2023), Tensor Parallelism (TP) (Shoeybi et al., 2019), and Pipeline Parallelism (PP) (Huang et al., 2019; Harlap et al., 2018; Narayanan et al., 2021; Qi et al., 2023; 2024). Each of these methods has its own advantages and limitations. For instance, ZeRO is effective in reducing memory by eliminating redundant parameter storage, but suffers from high communication overhead when gathering partitioned parameters and gradients for scenarios with limited network bandwidth or requiring frequent parameter updates. TP can efficiently handle large model parameters by splitting them across devices, but often faces low arithmetic intensity and requires significant inter-device communication. Among these techniques, PP shows distinct advantages due to its low communication cost and high arithmetic intensity, making it particularly attractive for training large-scale models. However, PP faces two significant challenges: pipeline bubbles and high memory consumption. Pipeline bubbles occur when there are idle periods in the pipeline stages, leading to suboptimal utilization of computational resources. Various strategies have been proposed to mitigate pipeline bubbles, such as token-level PP (Li et al., 2021) and interleaved 1F1B (Narayanan et al., 2021). An exceptional advancement is zero-bubble pipeline parallelism (Qi et al., 2023; 2024), which achieves almost zero bubble in many scenarios through splitting backward pass into activation gradient computation and weight gradient computation. In most PP schedules, the activations of several microbatches are stored to reduce pipeline bubbles, making memory a critical bottleneck to scale large models. Previous work has explored activation recomputation (Chen et al., 2016; Korthikanti et al., 2023), memory transferring (Kim et al., 2023) and memory-efficient V-Shape scheduling (Qi et al., 2024) to mitigate this issue. Despite various effort, the memory bottleneck still poses the largest limitation for PP.

In this paper, we focus on an imbalance issue in PP caused by vocabulary-related layers, which is often overlooked in practice but can significantly degrade the performance in both throughput and memory. Typically, transformer layers are uniformly distributed across pipeline stages, while the first stage contains an additional input layer and the last stage contains an additional output layer. Such imbalanced setup greatly hurts the performance in both computation

[†] Work was done during an internship at Sea AI Lab. [1]Sea AI Lab [2]National University of Singapore. Correspondence to: Xinyi Wan <wanxy@sea.com>.

Proceedings of the 5th MLSys Conference, Santa Clara, CA, USA, 2024. Copyright 2024 by the author(s).

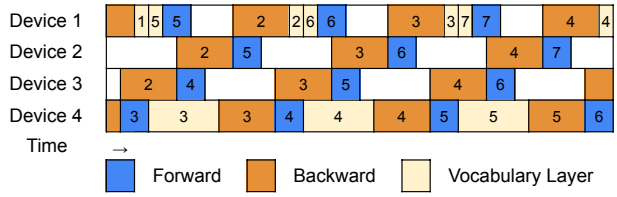


Figure 1. Repeating pattern in an imbalanced pipeline. Bubbles are incurred due to an extra output layer in the last pipeline stage.

and memory. Firstly, pipeline bubbles are introduced in other pipeline stages due to their less workload, as shown in Figure 1. Additionally, the additional input layer in the first stage exacerbates the memory bottleneck for most PP schedules like 1F1B (Harlap et al., 2018). Finally, as the vocabulary size grows larger (Tao et al., 2024), this imbalance becomes more pronounced, as shown in Figure 2. For instance, in the case of Gemma2 9B with a vocabulary size of 256k (Team et al., 2024), both the computation and parameter memory of the output layer are approximately 5 times those of the transformer layers, highlighting the severity of the issue.

To address this imbalance issue, we propose a novel Vocabulary Parallelism approach to balance the computation and memory in PP. By partitioning the vocabulary layers across all pipeline devices, we introduce several methods to group the computation and communication barriers together with a generalizable scheduling approach in PP, with only a small constant memory overhead. Extensive experiments demonstrate our approach significantly outperforms naïve layer redistribution and other existing methods, resulting in up to 51% improvement in throughput.

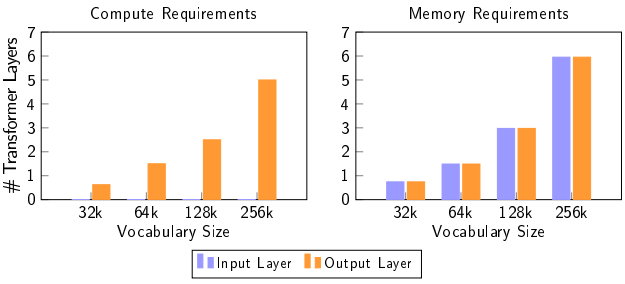


Figure 2. Ratio of compute and memory of vocabulary layers compared to transformer layers in Gemma2-9B.

2 Related Work

Balancing Activation Memory A line of research addresses another aspect of imbalance in PP, the activation memory with the 1F1B schedule. For instance, BPipe (Kim et al., 2023) transfers activations between devices, trading communication for reduced peak memory. Another approach uses V-Shape scheduling to create a pipeline schedule with balanced and efficient memory usage. These methods are orthogonal to our work, and combining them can achieve fully balanced pipeline parallelism in both computation and memory (activations and parameters).

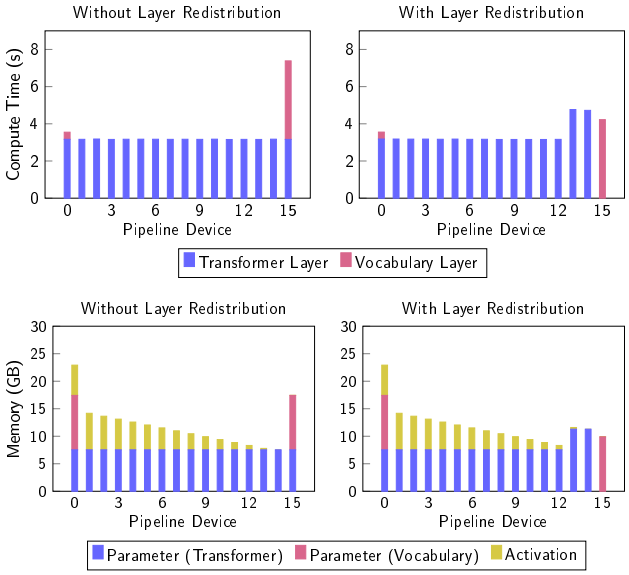


Figure 3. Transformer Layer Redistribution for a 7B GPT-like model with vocabulary size 128k. In this case, each stage has 2 transformer layers, while output layer is equivalent to 2.4x of transformer layer on compute and 2.6x on parameter memory.

Balancing Vocabulary Layers Some existing training systems try to mitigate the imbalance caused by vocabulary layers by redistributing transformer layers across different stages. DeepSpeed (Smith et al., 2022) uses a greedy method to automatically re-balance the workload between stages at the layer level. Similar strategies are employed in the training of Skywork-MoE (Wei et al., 2024). However, simply redistributing transformer layers faces several disadvantages. Firstly, even after redistribution, compute imbalance can still persist since only a subset of pipeline stages receive additional layers. An example is shown in Figure 3. This is particularly evident when the number of layers on each stage is small. Secondly, different layer types have varying compute-to-memory ratios, meaning that the re-balancing can only be based on either compute or memory but not both. In practice, the re-balancing is typically performed based on compute, leaving the memory imbalance still significant, particularly for input vocabulary layers that require minimal compute but substantial memory. Lastly, the effectiveness and planning of redistribution heavily depend on both the model settings and pipeline parallel settings. This makes it less flexible and challenging to adopt in various scenarios.

It's also worth noting that some models pretrained from scratch like Llama 3 (Dubey et al., 2024) reduce one transformer layer each from the first and the last stages, respectively. This method requires changes to architecture of models, which is out of the scope of this paper. Also, it has limitations if the training starts from a checkpoint where the number of transformer layers of model is fixed.

Another method to mitigate the imbalance problem in pipeline parallelism (PP) is the interlaced pipeline, reported by the automatic parallelism framework nnScaler (Lin et al., 2024). This approach distributes the input and output vocabulary layers across different pipeline devices using a tensor parallel (TP) style (Narayanan et al., 2021). By alternating between TP for vocabulary layers and PP for transformer layers, it aims to balance compute and memory overhead. However, TP requires frequent synchronization between devices, leading to two major drawbacks. First, the peak activation memory for 1F1B increases to 1.5 times of its original value (see Appendix B.1), which may make the critical memory bottleneck even worse. Second, the synchronized all-reduce during the output vocabulary layer introduces additional pipeline bubbles for each microbatch. Our ablation study in Appendix B.2 shows these all-reduce along slows down the end to end training by approximately 11% on 32 GPUs. These significant overheads in both activation memory and pipeline bubbles render the interlaced pipeline impractical in real-world scenarios.

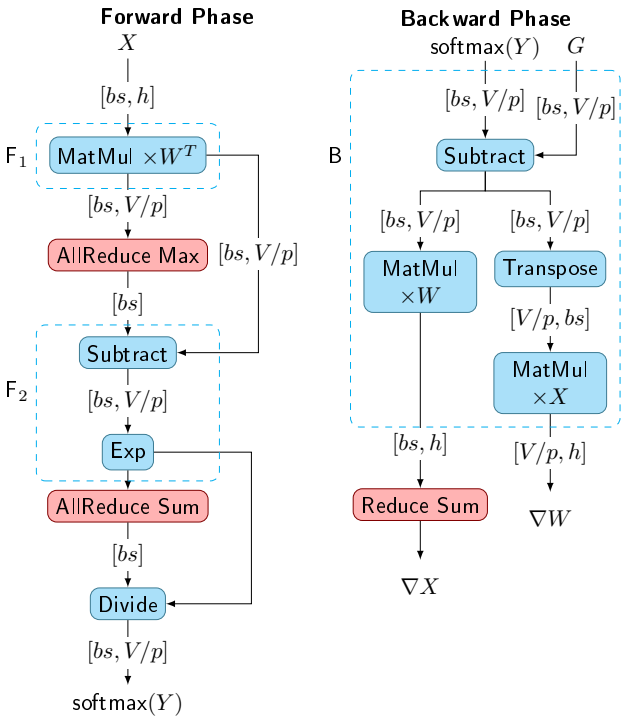


Figure 4. Computation graph of the output layer after partitioning across the vocabulary dimension. There are three all-reduce / reduce communications across all devices.

3 Vocabulary Parallelism in Pipeline Parallelism

To completely address the imbalance issue in PP, we propose Vocabulary Parallelism under the following design principles:

- We partition the vocabulary layers across the vocabulary dimension, and distribute it evenly to all pipeline devices.
- To be native to pipeline parallelism, the computation of vocabulary layers should be represented as passes similar to forward/backward passes of transformer layers.
- Integrating vocabulary passes into pipeline schedules should not drastically affect the memory and efficiency of the original pipeline schedule.

Intuitively, after partitioning the vocabulary layers to all pipeline devices, computations on each device can be scheduled independently by inserting them cleverly into the existing pipeline passes, as long as the dependencies are still satisfied. However, it is worth noting that partitioning the softmax computation creates several all-reduce operations. These communication barriers create cross-device dependencies.

Driven by this intuition, in Section 4, we discuss how to partition the computation in vocabulary layers to multiple devices and group them into pipeline passes. We observe that the communication barriers (e.g. the all-reduces in softmax) within the vocabulary layers increases the activation memory consumption of the pipeline schedule. As an improvement, we propose two novel algorithms to reduce the number of communication barriers, which reduces the activation memory overhead to minimum.

In Section 5, we discuss how to integrate these vocabulary passes into existing pipeline schedules. Inspired by the framework presented in Qi et al. (2024), we insert the vocabulary passes into the building blocks of existing schedules and simply repeat building blocks to construct pipeline schedules. This relieves us from the hassle of deciding the ordering of vocabulary passes of every microbatch and is naturally generalizable to other schedules.

4 Vocabulary Passes Construction

In this section, we introduce how to split the vocabulary layers into several computation passes after partitioning

them across all pipeline devices, and how to optimize the number of communication barriers.

4.1 A Naïve Approach

In the input layer, each device can perform forward and backward computations independently. We provide details on the input layer in Appendix C, and focus on the output layer for the remainder of this paper.

Figure 4 shows the computation graph of the output layer after partitioning the layer across p devices. We denote the microbatch size as b, sequence length as s, hidden dimension as h and vocabulary size as V.

The partitioned output layer can be grouped into three computation passes F_1, F_2 and B, separated by three all-reduce / reduce communications involving the maximum of logits, the sum of logits and the input gradients respectively. We can overlap these all-reduce communications with transformer layer computation by placing them in a separate stream, as shown in Figure 5.

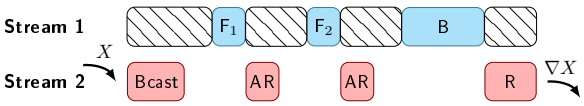


Figure 5. Overlapping all-reduce communication with transformer layer computation.

Figure 6 shows the computation and communication dependencies for a single microbatch. Notably, each of these all-reduce communications will introduce a communication barrier across all pipeline devices, which complicates the pipeline scheduling. As shown later in Section 5.2, the number of communication barriers also increases the activation memory consumption of the pipeline schedule. Therefore, we aim to reduce the number of communication barriers by reordering the operations in the output layer.

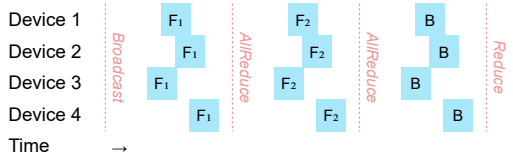


Figure 6. Scheduling dependencies in the naïve output layer implementation.

4.2 Definitions

We detail the computation in the output layer. Given the output of the last transformer layer X and the embedding weights W, we first compute Y:

$$Y = XW^T \tag{1}$$

Then, the (safe) softmax of each sequence is computed as follows:

$$\text{softmax}(Y_{ij}) = \frac{e^{Y_{ij}-m_i}}{\text{sum}_i} \tag{2}$$

where $m_i = \max_k Y_{ik}$ is the maximum of the logits and $\text{sum}_i = \sum_k e^{Y_{ik}-m_i}$ is the sum of logit exponents.

Assuming the cross entropy loss is used, in the backward phase, we have

$$\nabla X = (\text{softmax}(Y) - G)\, W \tag{3}$$

$$\nabla W = (\text{softmax}(Y) - G)^T\, X \tag{4}$$

where G is the ground truth matrix with $G_{ig_i} = 1$ and $G_{ij} = 0$ otherwise, where g_i is label for token i.

4.3 Forward Phase Optimization

Inspired by online softmax (Milakov & Gimelshein, 2018; Dao et al., 2022), we observe that the all-reduce communication for m_i and sum_i can be done after computing the softmax. Instead of using the global maximum and sum, each device instead computes $\text{softmax}'(Y_i)$ using the local maximum and sum from its vocabulary partition. We then have

$$\text{softmax}(Y_{ij}) = \text{softmax}'(Y_{ij}) \times \frac{\text{sum}'_i \times e^{m'_i - m_i}}{\text{sum}_i} \tag{5}$$

where m'_i and sum'_i are the locally computed maximum and sum, respectively.

Using equation 5, we have Algorithm 1 that reduces the 3 communication barriers to 2, which are denoted as C_1 and C_2 respectively.

Algorithm 1 Output layer with 2 communication barriers

function forward_and_backward(W)
  $X \leftarrow$ Receive Broadcast ◁ C_0
  $Y \leftarrow XW^T$
  $m'_i \leftarrow \max_{k=1}^{V/p} Y_{ik}$
  $\text{sum}'_i \leftarrow \sum_{k=1}^{V/p} e^{Y_{ik}-m'_i}$ ◁ S
  $\text{softmax}'(Y_{ij}) \leftarrow \dfrac{e^{Y_{ij}-m'_i}}{\text{sum}'_i}$
  $m_i \leftarrow$ AllReduce m'_i
  $\text{sum}'_i \leftarrow \text{sum}'_i \times e^{m'_i - m_i}$ ◁ C_1
  $\text{sum}_i \leftarrow$ AllReduce sum'_i
  $\text{softmax}(Y_i) \leftarrow \text{softmax}'(Y_i) \times \dfrac{\text{sum}'_i}{\text{sum}_i}$
  $\nabla X' \leftarrow (\text{softmax}(Y) - G)W$ ◁ T
  $\nabla W \leftarrow (\text{softmax}(Y) - G)^T X$
  $\nabla X \leftarrow$ Reduce $\nabla X'$ ◁ C_2
end function

The elementwise operations in C_1 only involves tensors of size $[bs]$ as opposed to size $[bs, V/p]$, which greatly reduces the computation pressure when overlapped with transformer layer computation.

4.4 Backward Phase Optimization

We also observe that all three all-reduce / reduce communications can be done after computing the matrix multiplications for the input gradients. Note that

$$\nabla X = \text{softmax}'(Y)W \times \frac{\text{sum}'_i \times e^{m'_i - m_i}}{\text{sum}_i} - GW \quad (6)$$

We can compute $\text{softmax}'(Y)W$ and GW beforehand, and all-reduce ∇X after we obtain m_i and sum_i. Since the matrix multiplications in equation 6 are already computed, computing ∇X within the communication barrier only involves lightweight operations.

This allows us to complete both phases in the output layer with only a single communication barrier C_1, as shown in Algorithm 2.

Algorithm 2 Output layer with 1 communication barrier

function forward_and_backward(W)
  $X \leftarrow$ Receive Broadcast ◁ C_0
  $Y \leftarrow XW^T$
  $m'_i \leftarrow \max_{k=1}^{V/p} Y_{ik}$
  $\text{sum}'_i \leftarrow \sum_{k=1}^{V/p} e^{Y_{ik} - m'_i}$
  $\text{softmax}'(Y_{ij}) \leftarrow \frac{e^{Y_{ij} - m'_i}}{\text{sum}'_i}$ ◁ S
  $A \leftarrow \text{softmax}'(Y)W$
  $B \leftarrow GW$
  $m_i \leftarrow$ AllReduce m'_i
  $\text{sum}'_i \leftarrow \text{sum}'_i \times e^{m'_i - m_i}$
  $\text{sum}_i \leftarrow$ AllReduce sum'_i ◁ C_1
  $\nabla X \leftarrow$ Reduce $A \times \frac{\text{sum}'_i}{\text{sum}_i} - B$
  $\text{softmax}(Y) \leftarrow \text{softmax}'(Y) \times \frac{\text{sum}'_i}{\text{sum}_i}$
  $\nabla W \leftarrow (\text{softmax}(Y) - G)^T X$ ◁ T
end function

Note that the weight gradient step T can be arbitrarily delayed since no other operations depend on it, similar to the idea in zero-bubble strategy (Qi et al., 2023).

We compare the two algorithms with the naïve implementation in Figure 7. By placing the operations in the communication barrier in a separate stream, they will be able to overlap with transformer layer computation. Compared to Algorithm 1, Algorithm 2 introduces a bit more computation overhead (shown in Section 6.5). However, it is still beneficial to reduce the number of communication barriers. As shown in section 5.2, a reduction in the number of communication barriers will help to save activation memory.

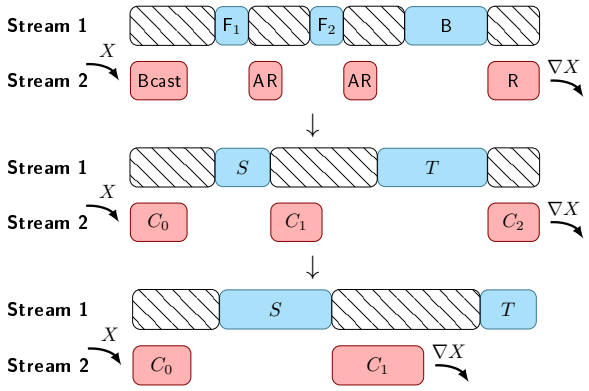


Figure 7. Computation order in the output layer for a single microbatch, corresponding to the naïve implementation, Algorithm 1 and Algorithm 2 respectively.

5 Pipeline Scheduling

In this section, we show a systematic method about how to make minimal changes to typical pipeline schedules to include the output layer passes. We apply our method on two different schedules, 1F1B (Harlap et al., 2018) and *V-Half* (Qi et al., 2024). Despite its popularity, an inherent problem of the 1F1B schedule is an imbalanced activation memory across pipeline devices. In contrast, *V-Half* balances out the activation memory by a V-Shape device placement, reducing the activation memory requirement to half of that of 1F1B. By integrating Vocabulary Parallelism into *V-Half*, we aim to achieve a completely memory-balanced pipeline.

5.1 Scheduling Dependencies

In Algorithms 1 and 2, we perform output layer computation with 2 and 1 communication barriers, respectively. For each microbatch, we have to integrate the output layer passes, S and T, into the pipeline schedules. The pipeline schedule has to adhere to the following constraints:

- All S passes must be scheduled after the forward pass of the last transformer layer completes.
- All T passes must be scheduled after all S passes complete.
- For Algorithm 1 only, the backward pass of the last transformer layer must be scheduled after all T passes complete. In contrast, the T passes can be arbitrarily delayed in Algorithm 2.

For example, Figure 8 shows the scheduling dependencies for a single microbatch in Algorithms 1 and 2 respectively.

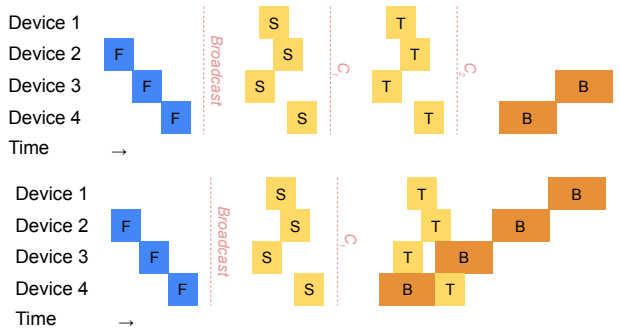


Figure 8. Scheduling Dependencies in Algorithms 1 and 2.

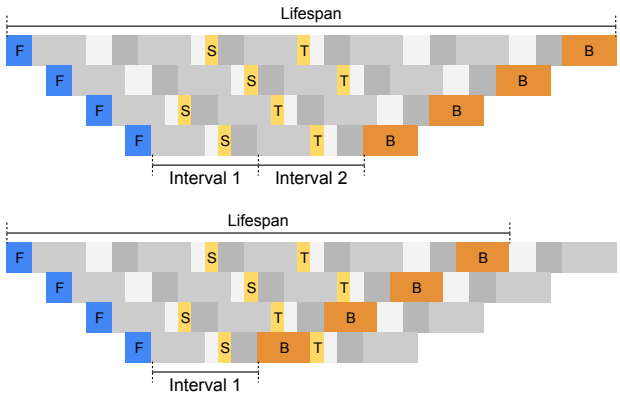


Figure 9. Modified building blocks for the 1F1B schedule corresponding to Algorithm 1 and Algorithm 2. The output layer passes are inserted.

5.2 Methodology

To elegantly integrate these S and T passes into typical pipeline schedules while adhering to the constraints, we follow Qi et al.'s framework (2024) to construct pipeline schedules. In this framework, each pipeline schedule can be structured by its building block, which is defined by the scheduling pattern of each microbatch. By uniformly repeating a building block, we can construct a pipeline schedule, with peak activation memory calculated by dividing its lifespan by its interval. The lifespan is the time between a forward and its corresponding backward, while the interval is the workload of a single microbatch on each device, as illustrated in Figure 9. This approach greatly simplifies dependency management for each microbatch and facilitates memory consumption analysis.

Considering the building block of the schedule, by inserting 2 or 1 intervals (for Algorithms 1 and 2 respectively) between the forward and backward pass of the last transformer layer, we can create space to schedule the output layer computation. Within the repeating interval in the building block, we can schedule output layer passes (S and T) arbitrarily in each pipeline device. We show an example based on 1F1B in Figure 9, where a *one-forward-one-backward-one-output* pattern is strictly followed. The final 1F1B schedules are presented in Figure 10, which is produced by uniformly repeating the building blocks. Additionally, the building block for *V-Half* can be found in Appendix D.

For the peak activation memory, as we insert at most 2 intervals to the lifespan, the peak activation memory only increases by at most 2 microbatches, which is a small constant overhead. This is a remarkable improvement compared to synchronous pipeline schedules, which multiplies the activation memory requirement by 1.5 (see Appendix B.1). Furthermore, the memory savings from balancing the vocabulary parameters outweighs the increase in activation memory.

Notably, as shown in Figure 9, the activation memory increased in terms of microbatches is equivalent to the number of communication barriers, which motivates our optimization of communication barriers in Section 4.

6 Experiments

We construct our experiments to verify: a) Our schedules with Vocabulary Parallelism can bring acceleration; b) Our methods can achieve a balanced memory usage when combined with memory-balanced schedules like *V-Half* (Qi et al., 2024); c) The partitioning of vocabulary layers has a reasonable scaling factor compared to linear scaling.

6.1 Implementation

We implement the pipeline scheduler and the partitioned vocabulary layers based on the open-source Megatron-LM project (Narayanan et al., 2021).

Scheduling under the assumption that backward takes twice time as forward might introduce unnecessary bubbles, especially when these values differ significantly. As a result, we profile the run time of the passes and schedule the S and T passes accordingly[1].

We handle the communication groups in separate streams, allowing the communication barrier to overlap with the transformer layer passes. We map the CUDA streams to separate GPU work queues to achieve this overlapping[2].

[1]The profiling verifies whether the backward pass runs approximately twice the time as the forward pass (unrelated to the vocabulary layer), in order to insert the vocabulary passes appropriately. The pipeline schedule generated will remain unchanged unless this ratio deviates by a certain threshold. In practice, we find that the difference is negligible in most transformer networks, and these differences would not change the pipeline schedule. Hence, this profiling could be viewed as optional.

[2]See https://docs.nvidia.com/deploy/mps/index.html#cuda-device-max-connections.

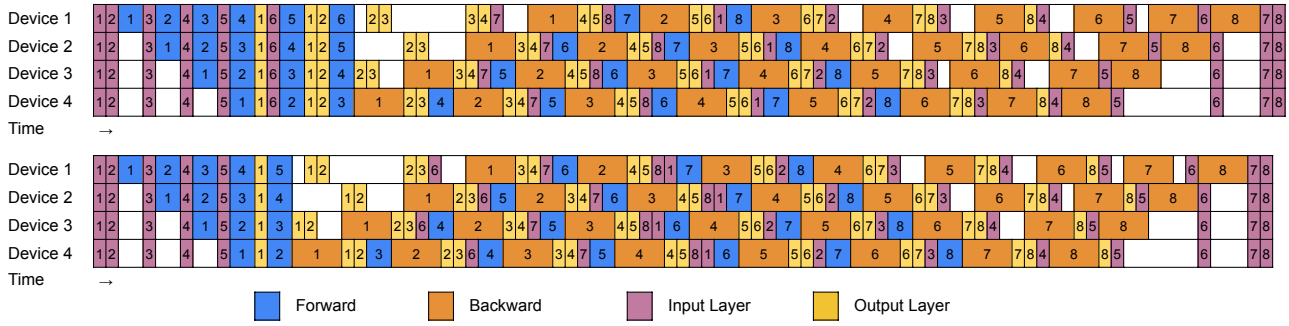


Figure 10. Full 1F1B schedules with Vocabulary Parallelism, corresponding to (a) Algorithm 1 and (b) Algorithm 2 respectively. Algorithm 1 requires activation memory for $p + 2$ microbatches while Algorithm 2 only requires $p + 1$, where p is the number of devices.

However, this would affect communication-computation overlap performance of tensor parallelism (Narayanan et al., 2021) as it relies on single work queues. To mitigate this problem, we set all model parallel communication groups to use high-priority streams. Additionally, both AllReduce and Reduce mentioned on Algorithm 1 and 2 are implemented as NCCL (NVIDIA) AllReduce to avoid imbalanced communication volume across devices.

We also pad the vocabulary size to be a multiple of $2p$ to improve memory alignment in the vocabulary layers, where p is the number of devices. In particular, we observe an approximate 8% increase in performance if our method is applied to 24 devices with padded size 256032 (a multiple of 48), compared to the original value 256008.

Note that our method makes tying input and output embedding weights easier as the input and output embedding weights now have the same device placement and can use the shared weight tensor. This saves GPU memory and avoids the additional all-reduce to synchronize gradients. However, in all our experiments, we adapted the more difficult setting, untying the input and output embedding weights, since it is adapted by some open source models like Llama 3 (Dubey et al., 2024).

6.2 Setup

We compare the following methods implemented on the 1F1B schedule (Harlap et al., 2018).

- Baseline: The naïve implementation in Megatron-LM. It distributes the transformer layers equally to all pipeline stages, while assigning the input and output layers to the first and last pipeline devices. This leads to highly imbalanced compute and memory.
- Redis: Redistributes the transformer layers across pipeline stages to balance out the computation as much as possible. We follow the derivation by Narayanan et al. (2021) to estimate the number of floating point operations in each pipeline stage, and minimize the length of the longest stage.
- Vocab-1: Implements Vocabulary Parallelism with only forward phase optimization, as described in Algorithm 1.
- Vocab-2: On top of Vocab-1, applies backward phase optimization, as described in Algorithm 2.
- Interlaced: Our implementation of the fully synchronous interlaced pipeline proposed by Lin et al. (2024).

Table 1. Settings used in experiments on 1F1B schedule.

PIPELINES (GPUS)	8	16	32
MODEL SIZE	≈ 4B	≈ 10B	≈ 21B
LAYERS	32	48	64
ATTENTION HEADS	24	32	40
HIDDEN SIZE	3072	4096	5120
SEQUENCE LENGTH	2048 / 4096		
MICROBATCH SIZE	1		
NUMBER OF MICROBATCHES	128		
VOCABULARY SIZE	32K / 64K / 128K / 256K		

We experiment the implementations by pretraining GPT-like models of varying model and vocabulary sizes with up to 32 NVIDIA A100 SXM 80G GPUs distributed across 4 nodes, interconnected by a RoCE RDMA network. The running time of each iteration is recorded after several warm-up iterations. We compare the 5 methods under each fixed setting of model and vocabulary size, shown in Table 1.

Our experiments use pure pipeline parallelism to verify that our method improves pipeline parallelism as expected. Given that our method is orthogonal to tensor and data parallelism, the conclusions can be generalized to the production environment where pipeline parallelism is used together with tensor and data parallelism.

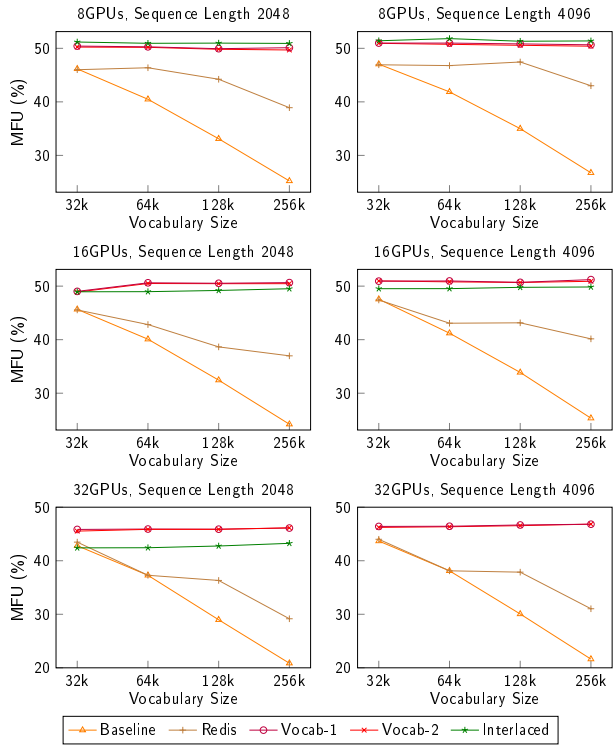


Figure 11. Throughput of different methods on 1F1B. Interlaced OOMs when training with 32 GPUs and sequence length 4096.

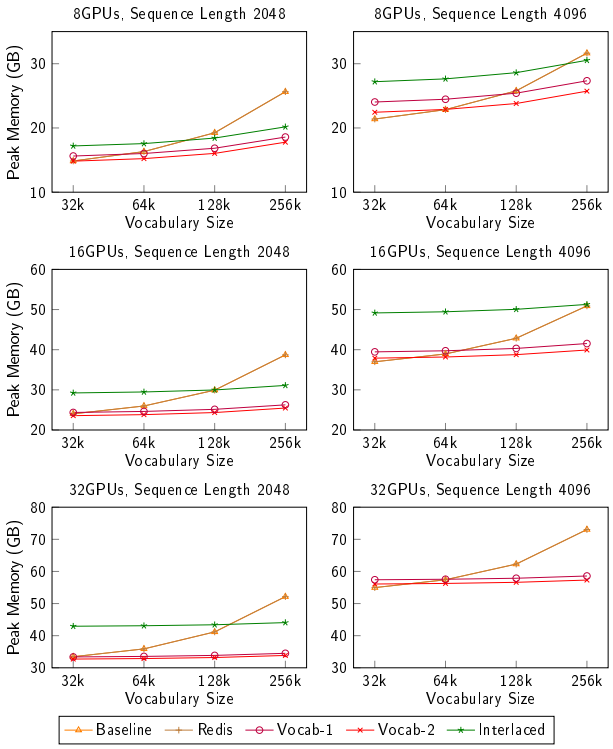


Figure 12. Peak memory of different methods on 1F1B

6.3 Comparison of Methods

We present comparisons of the throughput measured in FLOPs utilization (MFU) and peak memory in Figures 11 and 12, respectively. As shown in the figures, the layer redistribution approach suffers from a significant performance degradation of 8% to 33% for large vocabulary sizes, since the output layer alone already has a higher computation cost than that in the other pipeline devices. Its performance is also highly dependent on the model configuration, or more specifically, the ratio of compute between the vocabulary layer and transformer layers. For example, there is a 9.7% drop in MFU when increasing the vocabulary size from 64k to 128k for the 10B model with sequence length 2048, but that is not observed with sequence length 4096. In contrast, the Vocab and Interlaced approaches have a consistent MFU when scaling up the vocabulary sizes. Vocabulary Parallelism outperforms the interlaced pipeline under a multi-node setup, due to its overlapped communication. For the 21B model, Vocabulary Parallelism outperforms the interlaced pipeline by 6.7% to 8.2% in MFU.

For peak memory usage, the naïve implementation and layer redistribution approaches have an imbalanced parameter memory, leading to high peak memory for large vocabulary sizes. Although the Vocabulary Parallelism methods require extra activation memory, it is effectively negligible when we scale up the pipeline parallel size. However, the interlaced pipeline requires 1.5 times activation memory compared to 1F1B. This resulted in out-of-memory when training the 21B model with sequence length 4096.

6.4 Memory-Balanced Schedule

We show that our method can achieve a balanced memory usage by applying Vocab-1 on the *V-Half* schedule (Qi et al., 2024), a memory-balanced schedule. The implementation is based on the open-sourced *V-Half* implementation by Qi et al. (2024). To support division into virtual pipeline chunks, we adopt different configurations in the experiments, as shown in Table 2.

Table 2. Settings used in experiments on *V-Half* schedule.

Pipelines (GPUs)	16	24	32
Model Size	≈ 7B	≈ 16B	≈ 30B
Layers	32	48	64
Attention Heads	32	40	48
Hidden Size	4096	5120	6144
Sequence Length	2048 / 4096		
Microbatch Size	1		
Number of Microbatches	128		
Vocabulary Size	32k / 64k / 128k / 256k		

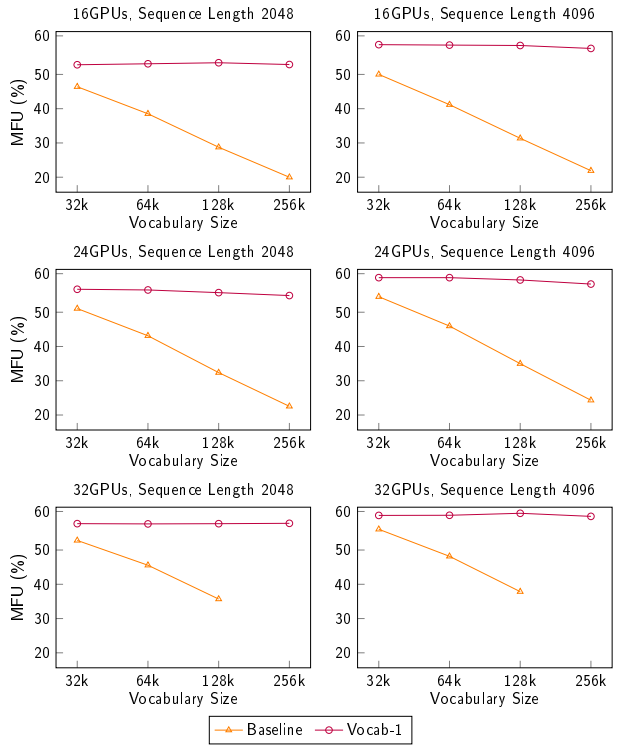


Figure 13. Throughput of different methods on *V-Half*. Baseline OOMs when training with 32 GPUs and vocabulary size 256k.

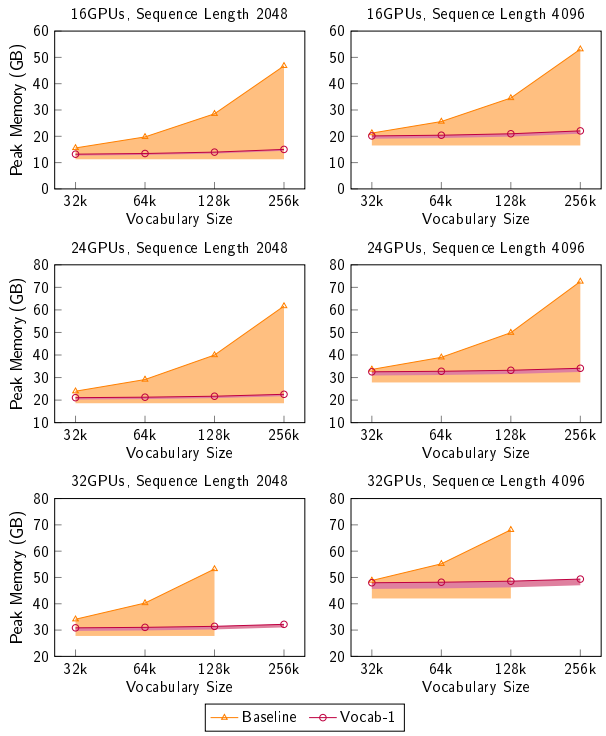


Figure 14. Peak memory of different methods on *V-Half*. The shaded area denotes the range of maximum allocated memory for all devices.

We compare the naïve *V-Half* schedule implementation and that incorporated with Vocab-1. The throughput and peak memory for each pipeline device are shown in Figures 13 and 14 respectively. The naïve implementation resulted in out-of-memory in cases with 32 GPUs and a 256k vocabulary size.

Similar to the previous experiments, the baseline suffers from a huge performance drop when we increase the vocabulary size, while Vocab-1 maintains a steady MFU, consistently outperforming the baseline by 7.2% to 143%. Besides, the baseline has a significant memory imbalance across pipeline devices with up to 45GB difference, while Vocab-1 achieves a balanced memory usage across pipeline devices. Although the first pipeline device still holds slightly more parameters due to positional and token type embedding, the extra memory required is a small constant. In our experiments, this is less than 2.5GB.

6.5 Scaling Analysis of Vocabulary Layers

We analyze the scalability of vocabulary layers in our Vocabulary Parallelism implementation. Using a vocabulary size of 256k, we measure the average throughput of all S and T passes across all devices in our implementation. We compare this with the ideal scenario where the vocabulary layers linearly scale (i.e. p times of the original throughput when distributed to p devices).

The output layers for Vocab-1 and Vocab-2 are considered separately. The time used for the communications is not included since it overlaps with other computation. The results are shown in Table 3.

Table 3. The scaling factor of vocabulary layer computation relative to linear scaling on sequence lengths 2048 and 4096.

SEQ	LAYER	8GPU	16GPU	32GPU
2048	OUTPUT-VOCAB-1	91.29%	84.22%	80.59%
	OUTPUT-VOCAB-2	86.72%	79.84%	75.93%
	INPUT	39.99%	28.85%	15.18%
4096	OUTPUT-VOCAB-1	93.21%	88.02%	85.24%
	OUTPUT-VOCAB-2	88.36%	83.42%	79.66%
	INPUT	27.69%	15.52%	8.35%

Parallelizing the vocabulary layers comes with some computation overhead, which can be attributed to two causes. Firstly, partitioning the vocabulary layers will reduce the model FLOPs utilization (MFU) of GPU kernels as the operations are smaller and hence less parallelized. Secondly, this brings extra computation, especially for the input layer where all devices have to construct the output tensor, whose

size is independent of the size of the vocabulary partition. However, it's worth noting that both input and output still only take a small portion of the computation of the entire model after being partitioned.

7 Conclusion and Future Work

In this work, we identified the problem that when training LLMs with pipeline parallelism, vocabulary layers causes non-negligible imbalance for both compute and memory. Existing methods either fails to achieve a balance or introduce significant performance overhead to the original pipeline schedule. To address this issue, we proposed Vocabulary Parallelism, a method that partitions vocabulary layers evenly to pipeline devices and integrates them into existing pipeline schedules. Our method achieves compute and memory balance for the vocabulary layers. As a result, experiments shows that it improves the throughput by up to 51% while also reduces peak memory consumption compared to existing methods.

Although our implementation of the vocabulary layers are pure python-based, we find that similar optimizations to Algorithm 2 opens an opportunity of fusing the forward and backward pass in CUDA kernels to avoid writes/reads of the softmax results, which can be huge in long-context large-vocabulary settings, to main memory, similar to the rationale of FlashAttention (Dao et al., 2022). Also, while our work focuses on the imbalanced vocabulary layers for pure text-based LLMs, we believe the embedding layers for multimodal LLMs suffer from the same problem and can be further explored.

References

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.

Chen, T., Xu, B., Zhang, C., and Guestrin, C. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.

Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Harlap, A., Narayanan, D., Phanishayee, A., Seshadri, V., Devanur, N., Ganger, G., and Gibbons, P. Pipedream: Fast and efficient pipeline parallel dnn training. *arXiv preprint arXiv:1806.03377*, 2018.

Huang, Y., Cheng, Y., Bapna, A., Firat, O., Chen, D., Chen, M., Lee, H., Ngiam, J., Le, Q. V., Wu, Y., et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.

Kim, T., Kim, H., Yu, G.-I., and Chun, B.-G. Bpipe: Memory-balanced pipeline parallelism for training large language models. In *International Conference on Machine Learning*, pp. 16639–16653. PMLR, 2023.

Korthikanti, V. A., Casper, J., Lym, S., McAfee, L., Andersch, M., Shoeybi, M., and Catanzaro, B. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems*, 5, 2023.

Li, Z., Zhuang, S., Guo, S., Zhuo, D., Zhang, H., Song, D., and Stoica, I. Terapipe: Token-level pipeline parallelism for training large-scale language models. In *International Conference on Machine Learning*, pp. 6543–6552. PMLR, 2021.

Lin, Z., Miao, Y., Zhang, Q., Yang, F., Zhu, Y., Li, C., Maleki, S., Cao, X., Shang, N., Yang, Y., Xu, W., Yang, M., Zhang, L., and Zhou, L. nnScaler: Constraint-Guided parallelization plan generation for deep learning training. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 347–363, Santa Clara, CA, July 2024. USENIX Association. ISBN 978-1-939133-40-3.

Milakov, M. and Gimelshein, N. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.

Narayanan, D., Shoeybi, M., Casper, J., LeGresley, P., Patwary, M., Korthikanti, V., Vainbrand, D., Kashinkunti, P., Bernauer, J., Catanzaro, B., Phanishayee, A., and Zaharia, M. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–15, 2021.

NVIDIA. GitHub - NVIDIA/nccl: Optimized primitives for collective multi-GPU communication — github.com. https://github.com/NVIDIA/nccl. [Accessed 31-10-2024].

Qi, P., Wan, X., Huang, G., and Lin, M. Zero bubble pipeline parallelism. *arXiv preprint arXiv:2401.10241*, 2023.

Qi, P., Wan, X., Amar, N., and Lin, M. Pipeline parallelism with controllable memory. *arXiv preprint arXiv:2405.15362*, 2024.

Rajbhandari, S., Rasley, J., Ruwase, O., and He, Y. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–16. IEEE, 2020.

Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

Smith, S., Patwary, M., Norick, B., LeGresley, P., Rajbhandari, S., Casper, J., Liu, Z., Prabhumoye, S., Zerveas, G., Korthikanti, V., et al. Using deepspeed and megatron to train megatron-turing nlg 530b, a large-scale generative language model. *arXiv preprint arXiv:2201.11990*, 2022.

Tao, C., Liu, Q., Dou, L., Muennighoff, N., Wan, Z., Luo, P., Lin, M., and Wong, N. Scaling laws with vocabulary: Larger models deserve larger vocabularies. *arXiv preprint arXiv:2407.13623*, 2024.

Team, G., Riviere, M., Pathak, S., Sessa, P. G., Hardin, C., Bhupatiraju, S., Hussenot, L., Mesnard, T., Shahriari, B., Ramé, A., et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Wei, T., Zhu, B., Zhao, L., Cheng, C., Li, B., Lü, W., Cheng, P., Zhang, J., Zhang, X., Zeng, L., et al. Skywork-moe: A deep dive into training techniques for mixture-of-experts language models. *arXiv preprint arXiv:2406.06563*, 2024.

Zhao, Y., Gu, A., Varma, R., Luo, L., Huang, C.-C., Xu, M., Wright, L., Shojanazeri, H., Ott, M., Shleifer, S., et al. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277*, 2023.

A Quantitative Analysis of Vocabulary Layers

Following the calculations of Narayanan et al. (2021) and neglecting insignificant terms, we present the computational and memory expenses in relation to a single transformer layer in Table 4. In this table we denote the microbatch size as b, sequence length as s, hidden dimension as h and vocabulary size as V. It is worth noting that the activation memory is excluded from this analysis, as it typically has a transient nature for vocabulary layers.

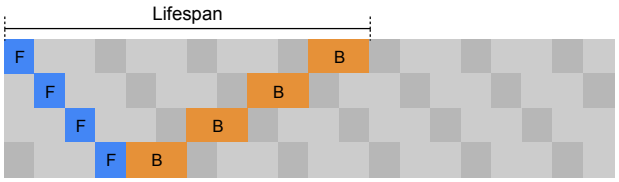
Table 4. Compute and memory cost of vocabulary and transformer layers

Layer Type	Compute FLOPs	Param Memory
Transformer	$bsh(72h + 12s)$	$24h^2$
Input	$3bsh$	$2hV$
Output	$6bshV$	$2hV$

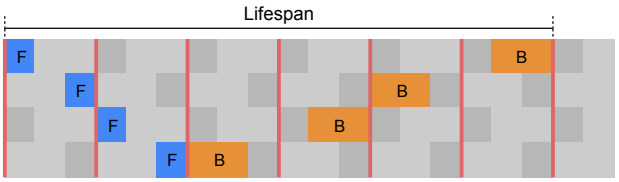
B More Analysis of Interlaced Pipeline

B.1 Memory Analysis

One concern of interlaced pipeline is that the peak activation memory of 1F1B is raised to 1.5 times of its original value. This increase in memory consumption can be analyzed using the framework introduced in Qi et al. (2024). As shown in Figure 15, the interlaced schedule enlarges the lifespan of 1F1B's building block from $3p$ to approximately $4.5p$ where p is the number of devices, resulting in 1.5x peak activation memory consumption.



(a) Building block of 1F1B



(b) Building block of interlaced pipeline parallel. The red vertical lines indicate synchronization introduced by TP of vocabulary layers.

Figure 15. Comparison between building blocks of 1F1B and Interlaced PP.

B.2 Overhead of Tensor Parallel Communication

In practice, tensor parallel is only used for intra-node parallelism due to its high communication volume. The interlaced pipeline has introduced a tensor parallel style parallelization for the vocabulary layers, which creates additional pipeline bubbles for each microbatch during the tensor parallel communication. To quantify the size of the bubbles, we conduct an ablation study by training a 21.5B model using 32 GPUs. We remove the synchronous all-reduce communications in the vocabulary layers, and measure the speedup in end-to-end iteration time. Note that the all-reduce communications that are overlapped with the computation are still kept.

By removing the synchronous all-reduce communications, the end-to-end iteration time improved by 10.95%. This shows that the synchronous all-reduce communications contributed to approximately 11% of the idle time when training using an interlaced pipeline. We conclude that the interlaced pipeline is undesirable for multi-node training.

C Vocabulary Parallelism for The Input Layer

While the output layers involve complex dependencies and communications, input layer computation can be completed independently before and after the transformer layer passes. The only required communications are an all-reduce communication after the forward pass, and a broadcast communication before the backward pass. These communications can be overlapped with the transformer layer computation, and can be scheduled well-ahead or after.

We schedule the input layer passes as follows:

- During the warm-up forward passes, we insert the input layer forward pass one microbatch before the first transformer layer forward pass. This allows time for gathering the input layer outputs.
- In the stable phase, we piggyback the input layer forward pass with the output layer passes, scheduled as least one repeating interval beforehand. Similarly, the input layer backward passes are piggybacked at least one repeating interval afterwards, allowing enough time to broadcast the output gradient to all devices.
- During the cool-down backward passes, we insert the input layer backward pass one microbatch after the last transformer layer backward pass.

This schedule ensures that each device is holding the input layer outputs for at most two microbatches at any instant, reducing the memory pressure.

D *V-Half* Pipeline Scheduling

Following the scheduling methodology in section 5.2, we show the building block for the *V-Half* schedule in Figure 16.

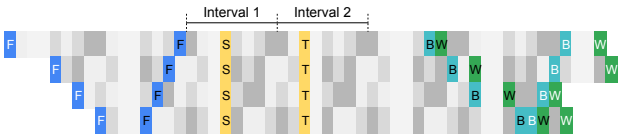


Figure 16. Modified building block for the *V-Half* schedule.

E Correctness Evaluation

Our implementation is based on the open-source Megatron-LM project (Narayanan et al., 2021). We compare the convergence curves of our implementation to that of the original Megatron-LM codebase to verify our implementation's correctness. The configurations follow the 4B model in section 6.2 with a vocabulary size of 256K, trained with 8 GPUs. Additionally, we verify that our implementation also works correctly with tensor parallelism, by using a pipeline parallel size of 4 and a tensor parallel size of 2.

The convergence curves are shown in Figure 17. It is shown that our implementation maintains correctness, albeit with some small numerical differences.

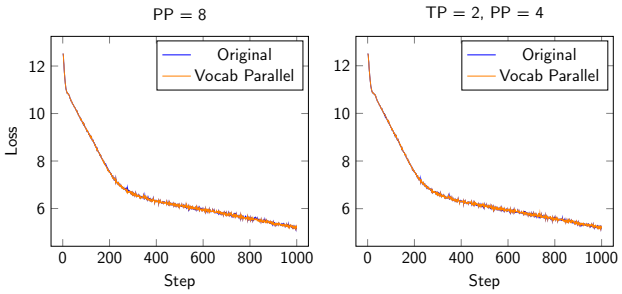


Figure 17. Convergence curves of our implementation against the original Megatron-LM codebase

F Detailed Experiment Data

For Sections 6.3 and 6.4, we present the detailed experimental data in Tables 5 and 6 respectively. The following metrics are computed:

- MFU: The FLOPs utilization of the training system. We follow Narayanan et al. (2021)'s derivation to compute the FLOPs of the model.
- Peak Memory: The maximum peak memory across all devices.

Table 5. Comparison of Methods on 1F1B.

Setup	Method	MFU (%)				Peak Memory (GB)			
		32k	64k	128k	256k	32k	64k	128k	256k
8GPU, Seq Length 2048	Baseline	46.16	40.48	33.11	25.23	14.86	16.32	19.25	25.64
	Redis	46.01	46.37	44.22	38.91	14.86	16.32	19.25	25.64
	Vocab-1	50.42	50.28	49.93	50.12	15.63	16.02	16.84	18.59
	Vocab-2	50.23	50.18	49.82	49.69	**14.83**	**15.23**	**16.04**	**17.78**
	Interlaced	**51.18**	**50.94**	**50.97**	**50.92**	17.20	17.57	18.43	20.17
8GPU, Seq Length 4096	Baseline	47.05	41.87	35.00	26.75	**21.39**	**22.85**	25.78	31.64
	Redis	46.93	46.78	47.44	43.01	**21.39**	**22.85**	25.78	31.64
	Vocab-1	50.98	50.98	50.83	50.66	24.04	24.47	25.41	27.34
	Vocab-2	50.93	50.75	50.56	50.40	22.44	22.89	**23.80**	**25.73**
	Interlaced	**51.41**	**51.82**	**51.32**	**51.38**	27.20	27.64	28.60	30.53
16GPU, Seq Length 2048	Baseline	45.66	40.09	32.44	24.21	24.03	25.98	29.92	38.71
	Redis	45.56	42.82	38.65	36.98	24.03	25.98	29.92	38.71
	Vocab-1	**49.02**	**50.62**	**50.54**	**50.66**	24.37	24.63	25.14	26.26
	Vocab-2	48.90	50.49	50.46	50.46	**23.57**	**23.83**	**24.35**	**25.47**
	Interlaced	48.94	48.97	49.19	49.52	29.23	29.47	29.97	31.10
16GPU, Seq Length 4096	Baseline	47.56	41.21	33.88	25.33	**36.99**	38.94	42.85	50.90
	Redis	47.41	43.07	43.15	40.15	**36.99**	38.94	42.85	50.90
	Vocab-1	50.93	**50.97**	**50.71**	**51.22**	39.46	39.73	40.31	41.53
	Vocab-2	**50.97**	50.80	50.68	50.90	37.89	**38.18**	**38.77**	**39.92**
	Interlaced	49.52	49.53	49.77	49.84	49.16	49.44	50.05	51.28
32GPU, Seq Length 2048	Baseline	42.81	37.28	28.97	20.86	33.45	35.89	41.17	52.16
	Redis	43.48	37.29	36.32	29.16	33.45	35.89	41.17	52.16
	Vocab-1	**45.85**	**45.92**	**45.90**	46.11	33.38	33.55	33.86	34.51
	Vocab-2	45.54	45.86	45.86	**46.16**	**32.72**	**32.88**	**33.20**	**33.84**
	Interlaced	42.40	42.43	42.75	43.25	42.94	43.09	43.40	44.07
32GPU, Seq Length 4096	Baseline	43.68	38.11	30.05	21.63	**54.97**	57.41	62.29	73.05
	Redis	44.01	38.12	37.87	31.03	**54.97**	57.41	62.29	73.05
	Vocab-1	**46.41**	**46.44**	**46.68**	46.83	57.41	57.56	57.88	58.58
	Vocab-2	46.23	46.35	46.55	**46.84**	56.09	**56.26**	**56.61**	**57.31**
	Interlaced	-	-	-	-	-	-	-	-

Table 6. Comparison of Methods on *V-Half*.

Setup	Method	MFU (%)				Peak Memory (GB)			
		32k	64k	128k	256k	32k	64k	128k	256k
16GPU, Seq Length 2048	Baseline	46.41	38.52	28.75	19.99	15.57	19.77	28.55	46.77
	Vocab-1	**52.82**	**53.11**	**53.41**	**52.89**	**13.20**	**13.46**	**13.98**	**15.02**
16GPU, Seq Length 4096	Baseline	50.01	41.17	31.36	21.90	21.22	25.61	34.56	53.11
	Vocab-1	**58.69**	**58.56**	**58.44**	**57.59**	**20.14**	**20.41**	**20.96**	**22.06**
24GPU, Seq Length 2048	Baseline	51.07	43.13	32.38	22.54	23.94	29.12	39.98	61.71
	Vocab-1	**56.70**	**56.50**	**55.72**	**54.86**	**21.08**	**21.29**	**21.72**	**22.57**
24GPU, Seq Length 4096	Baseline	54.53	45.96	34.99	24.31	33.60	38.97	49.90	72.60
	Vocab-1	**60.09**	**60.09**	**59.42**	**58.22**	**32.55**	**32.78**	**33.22**	**34.12**
32GPU, Seq Length 2048	Baseline	52.80	45.56	35.69	-	34.11	40.28	53.22	-
	Vocab-1	**57.70**	**57.62**	**57.69**	**57.80**	**30.85**	**31.04**	**31.42**	**32.18**
32GPU, Seq Length 4096	Baseline	56.06	48.17	37.85	-	48.84	55.19	68.12	-
	Vocab-1	**60.10**	**60.14**	**60.72**	**59.82**	**47.99**	**48.19**	**48.59**	**49.38**

G Artifact Appendix

G.1 Abstract

This section will outline the setup and experimental workflow of **Balancing Pipeline Parallelism with Vocabulary Parallelism** conducted on a single server equipped with 8 A100 GPUs.

G.2 Artifact check-list (meta-information)

- **Algorithm:** Vocabulary Parallelism, Online Softmax
- **Program:** Not used
- **Compilation:** Nvidia nvcc version 12.4, already in the container
- **Transformations:** Not used
- **Binary:** will be compiled on a target platform
- **Data set:** Customized C4 hosted in Huggingface
- **Binary:** will be compiled on a target platform.
- **Run-time environment:** Ubuntu 20.04.6. Needs docker. Requires root.
- **Hardware:** Server with 8 Nvidia A100 GPUs 80GB HBM.
- **Run-time state:** Server is idle.
- **Execution:** Takes at least 4 hours to complete.
- **Metrics:** Peak Memory, MFU
- **Output:** The data printed on console. The output of Quick Experiment is the key result.
- **Experiments:** Elaborated in the Installation and Experiment workflow sections.
- **How much disk space required (approximately)?:** 30 GB
- **How much time is needed to prepare workflow (approximately)?:** 30 minutes
- **How much time is needed to complete experiments (approximately)?:** 4 hours
- **Publicly available?:** Yes
- **Code licenses (if publicly available)?:** Apache License
- **Data licenses (if publicly available)?:** odc-by
- **Workflow framework used?:** No
- **Archived (provide DOI)?:**

G.3 Description

This experiment consists of 2 parts:

- **Quick Experiment** to quickly verify the result on a specific case.
- **Full Experiment** to run all cases on an 8-GPU server.

The **Full Experiment** employs the settings in table 7 same as the paper. The **Quick Experiment** focuses on a specific case where the sequence length is 4096 and the vocabulary size is 256k. We use Megatron-LM on which all methods are implemented to run training benchmarks.

Table 7. Artifact Settings used in experiments on 1F1B schedule.

Pipelines (GPUs)	8
Model Size	$\approx$ 4B
Layers	32
Attention Heads	24
Hidden Size	3072
Sequence Length	2048 / 4096
Microbatch Size	1
Number of Microbatches	128
Vocabulary Size	32K / 64K / 128K / 256K

G.3.1 How delivered

The code locates on https://github.com/sail-sg/VocabularyParallelism. The dataset will be automatically downloaded in experiment scripts.

G.3.2 Hardware dependencies

The tests should be conducted in a server with 8 Nvidia A100 GPUs, 80GB HBM.

G.3.3 Software dependencies

- **CUDA Driver Version**: 535.54.03
- **CUDA Version** 12.2
- **Docker**
- **NVIDIA Container Toolkit**

G.3.4 Data sets

The dataset is customized from C4 hosted in Huggingface https://huggingface.co/datasets/mtyeung/vocab_parallel_sample_dataset supporting various sequence length. It will be automatically downloaded in experiment scripts.

G.4 Installation

Run a container:

```
docker run --name vocab_torch24 \
    --network=host -d \
    --runtime=nvidia --gpus all \
    --ipc=host --ulimit memlock=-1 --ulimit
     stack=67108864 \
    --privileged=true \
    nvcr.io/nvidia/pytorch:24.03-py3 sleep
    infinity
```

Get inside the container, and clone the codes:

```
# Enter the container
docker exec -it vocab_torch24 bash
# Clone the codes
git clone https://github.com/sail-sg/
    VocabularyParallelism.git
cd VocabularyParallelism
```

Note that all the following commands should be run inside the `VocabularyParallelism` directory.

G.5 Experiment workflow

G.5.1 Quick Experiment

The quick experiment runs all the methods (*baseline*, *redis*, *interlaced*, *vocab-1*, *vocab-2*) on a specific setting in the paper:

- Sequence Length: 4096
- Vocabulary Size: 256k

The experiment will show 2 key results:

- **Peak Memory**
- **MFU**

Run all the methods one by one:

```
bash artifact/quick_exp.sh run baseline
bash artifact/quick_exp.sh run redis
bash artifact/quick_exp.sh run interlaced
bash artifact/quick_exp.sh run vocab-1
bash artifact/quick_exp.sh run vocab-2
```

This will automatically download the dataset from huggingface and run the training experiments. The log containing the result will locate in `quick-logs/<method>/stdout.log`. Each method should take around 6 minutes to complete.

Then run this to collect the results:

```
bash artifact/quick_exp.sh show-result
```

G.5.2 Full Experiment

This will run all experiments on a single server with 8 A100 GPUs.

The whole experiment will take around 3 hours to complete.

```
bash artifact/full_exp.sh artifact/
    exp_one_host.csv
```

Print results:

```
python artifact/show_result_full_exp.py
```

G.6 Evaluation and expected result

The output of **Quick Experiment** should show similar result as
https://github.com/sail-sg/VocabularyParallelism/blob/main/artifact/example-results/quick-exp.txt

The expected output of **Full Experiment** is located in
https://github.com/sail-sg/VocabularyParallelism/blob/main/artifact/example-results/full-exp.txt

The result should also roughly match the 2 rows in **Table 5. Comparison of Methods on 1F1B** in the paper:

- 8GPU, SEQ LENGTH 2048
- 8GPU, SEQ LENGTH 4096

The result shows that the throughput and peak memory of vocab-1 and vocab-2 are significantly better than baseline and redistribution. The throughput of interlaced is slightly better than vocab-1 and vocab-2. But the peak memory of interlaced is worse than our approach.

G.7 Experiment customization

User can customize the settings by changing the scripts quick_exp.sh, full_exp.sh, show_result_full_exp.py under under artifact/.